Sub-Pixel’s application and rethinking

马栋梁

2019-01-06

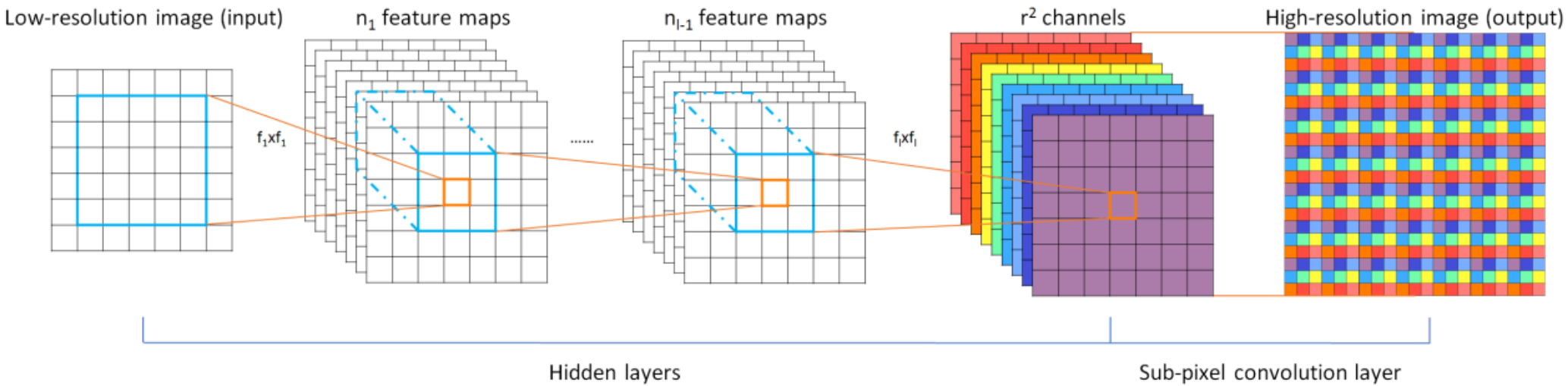


Figure 1. The proposed efficient sub-pixel convolutional neural network (ESPCN), with two convolution layers for feature maps extraction, and a sub-pixel convolution layer that aggregates the feature maps from LR space and builds the SR image in a single step.

Shi W, Caballero J, Huszar F, et al. Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network[J]. computer vision and pattern recognition, 2016: 1874-1883.

```python
class PixelShuffle(Module):

    def __init__(self, upscale_factor):
        super(PixelShuffle, self).__init__()
        self.upscale_factor = upscale_factor

    def forward(self, input):
        return F.pixel_shuffle(input, self.upscale_factor)

    def extra_repr(self):
        return 'upscale_factor={}'.format(self.upscale_factor)
```

```python
def pixel_shuffle(input, upscale_factor):
    r"""Rearranges elements in a tensor of shape :math:`[*, C*r^2, H, W]` to a
    tensor of shape :math:`[C, H*r, W*r]`.

    See :class:``torch.nn.PixelShuffle` for details.

    Args:
        input (Tensor): Input
        upscale_factor (int): factor to increase spatial resolution by

    Examples::

        >>> ps = nn.PixelShuffle(3)
        >>> input = torch.empty(1, 9, 4, 4)
        >>> output = ps(input)
        >>> print(output.size())
        torch.Size([1, 1, 12, 12])
    """
    batch_size, channels, in_height, in_width = input.size()
    channels //= upscale_factor ** 2

    out_height = in_height * upscale_factor
    out_width = in_width * upscale_factor

    input_view = input.contiguous().view(
        batch_size, channels, upscale_factor, upscale_factor,
        in_height, in_width)

    shuffle_out = input_view.permute(0, 1, 4, 2, 5, 3).contiguous()
    return shuffle_out.view(batch_size, channels, out_height, out_width)
```

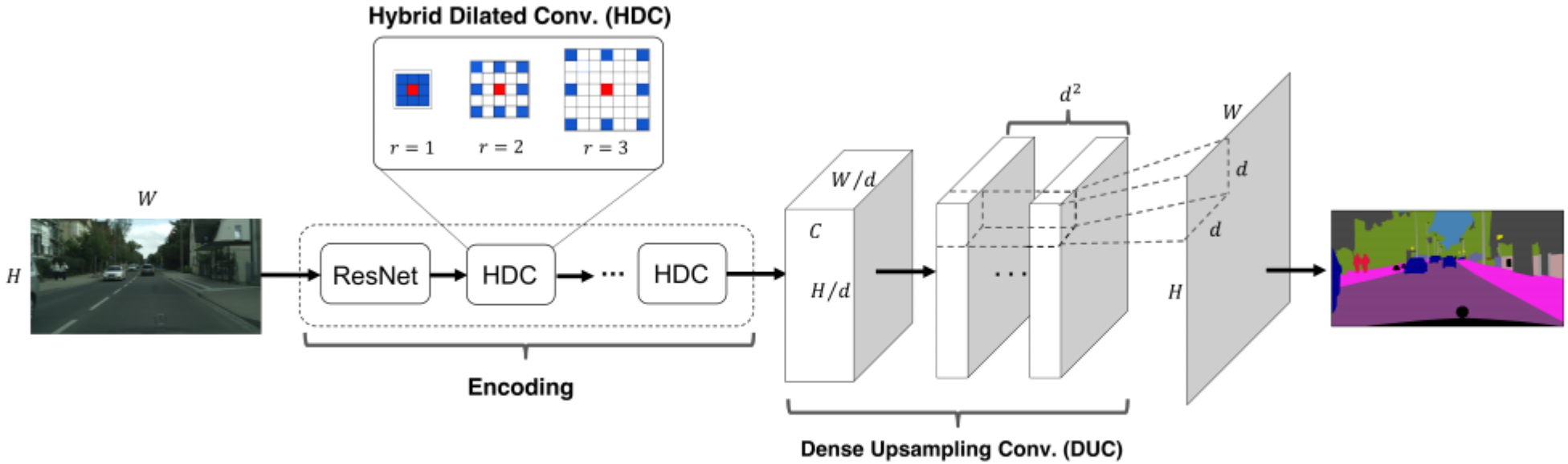


Figure 1. Illustration of the architecture of ResNet-101 network with Hybrid Dilated Convolution (HDC) and Dense Upsampling Convolution (DUC) layer. HDC is applied within ResNet blocks, and DUC is applied on top of network and is used for decoding purpose.

Wang P, Chen P, Yuan Y, et al. Understanding Convolution for Semantic Segmentation[J]. workshop on applications of computer vision, 2018: 1451-1460.

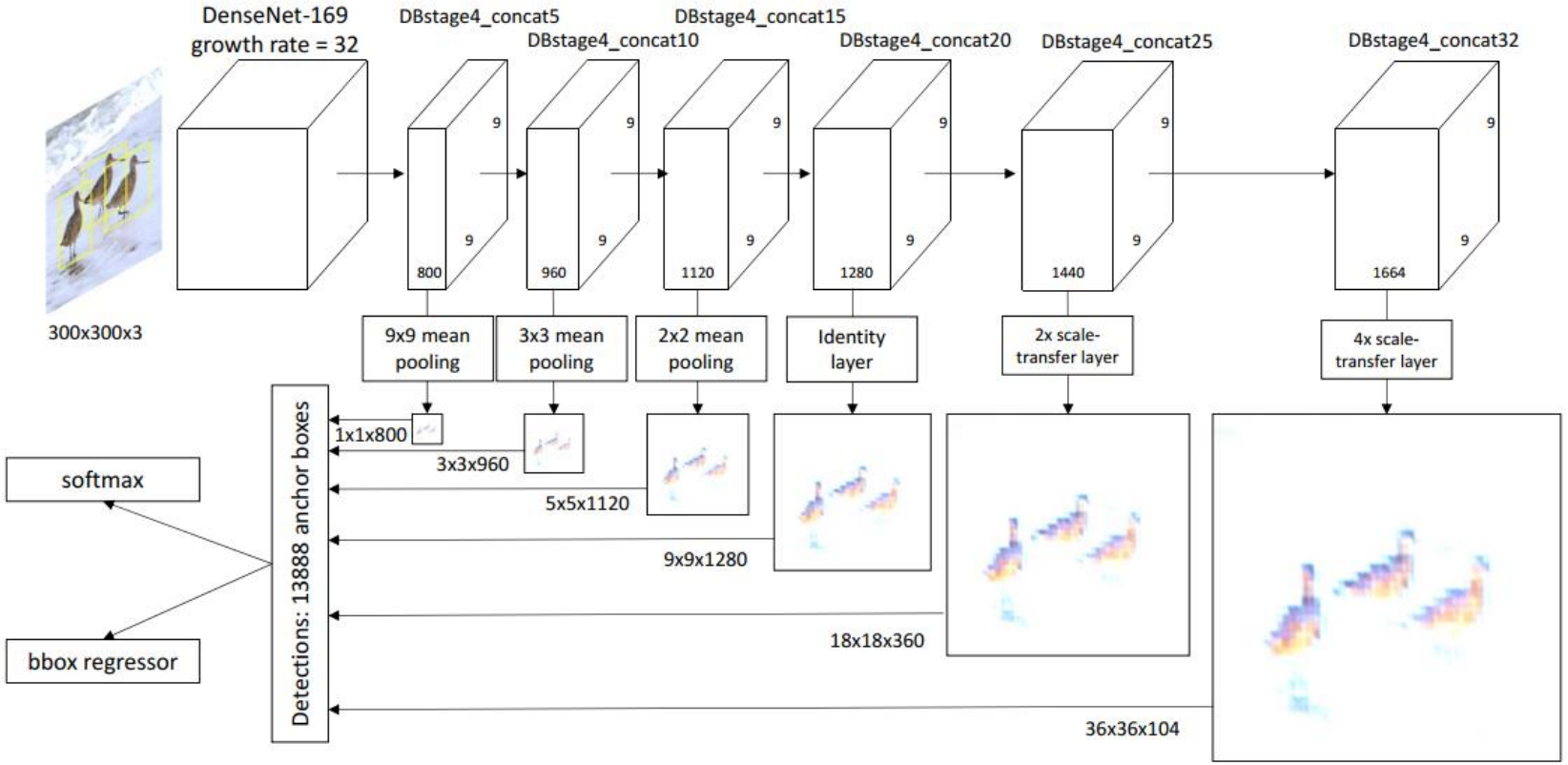


Figure 2. **Scale-Transferrable Detection Network (STDN) for detecting multi-scale objects in images.** We use DenseNet-169 [14] as the base network. The figure shows several layers in the last dense block of DenseNet-169. The output of each layer in dense block has the same height and width. We use scale-transfer layer to obtain high-resolution feature maps for detecting small objects, and use pooling layer to obtain feature maps with large receptive field for detecting large objects. These layers can be directly embedded into the base network with little computational overhead. The real-time performance of the detector is guaranteed by the scale-transfer module.

Zhou P, Ni B, Geng C, et al. Scale-Transferrable Object Detection[C]. computer vision and pattern recognition, 2018: 528-537.

Network	DS	ASPP	Augmentation	Cell	mIoU
Baseline	8	4	*yes*	*n/a*	72.3
Baseline	4	4	*yes*	*n/a*	70.9
DUC	8	*no*	*no*	1	71.9
DUC	8	4	*no*	1	72.8
DUC	8	4	*yes*	1	74.3
DUC	4	4	*yes*	1	73.7
DUC	8	6	*yes*	1	74.5
DUC	8	6	*yes*	2	74.7

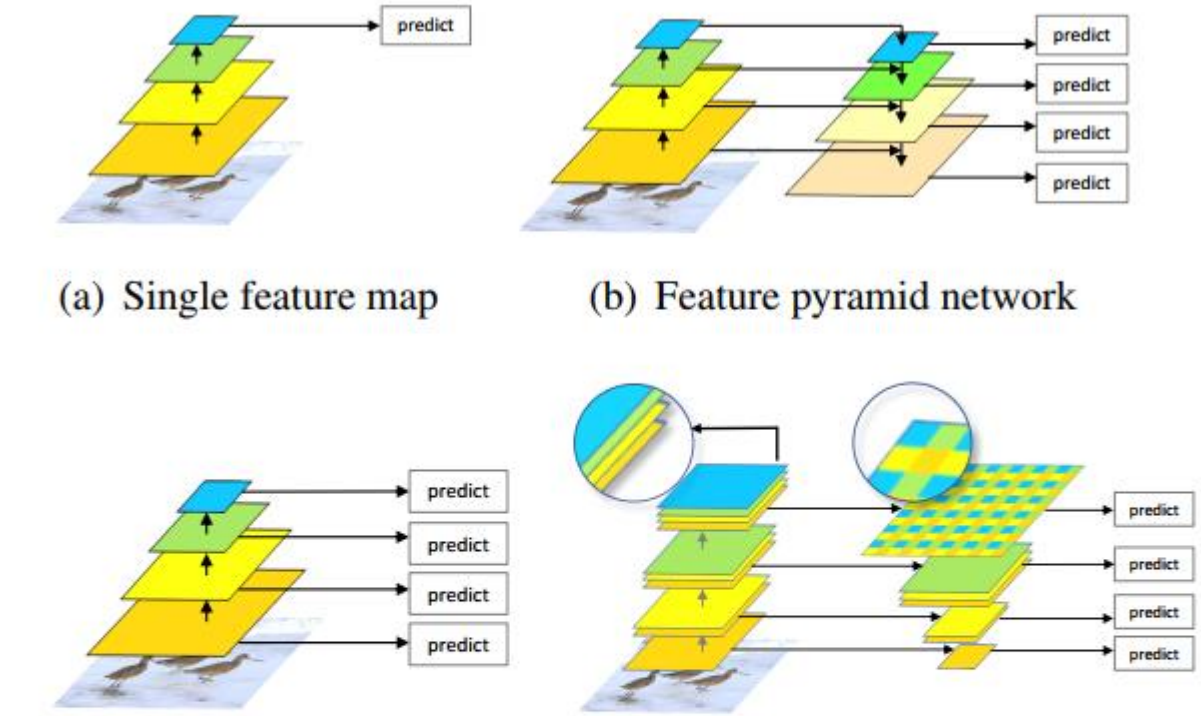
Table 1. Ablation studies for applying ResNet-101 on the Cityscapes dataset. **DS**: Downsampling rate of the network. **Cell**: neighborhood region that one predicted pixel represents.

Method	mAP	Anchor boxes	Input resolution
DenseNet-169 + SSD	73.8	14472	300×300
DenseNet-169 + STM	76.6	13888	300×300
DenseNet-169 + STM	76.6	13888	300×300
DenseNet-169 + stem block + STM	**78.1**	13888	300×300
Inception-v3 + STM	74.9	13888	332×332
DenseNet-169 + stem block + STM	**78.1**	13888	300×300

Table 3. **Ablation study on PASCAL VOC 2007 test set.** Note that DenseNet-169+SSD means replacing the base network of SSD from VGG to DenseNet-169. STM is the scale-transfer module. Stem block is explained in the above. For fair comparison, the input of Inception-v3 is resized to 332×332 so that the number of anchor box is consistent with the comparison test.

Reason

CNN is much smaller than the size of the input image. For example, for DenseNet-169 network, if the input image size is $300 \times 300 \times 3$, the size of the feature map on the top of the network will be $9 \times 9 \times 1664$, where 1664 is the number of channels. This means that one pixel in the feature map is related to about 33 pixels in the input image in the width and height space plane. It means that the CNN compresses the $33 \times 33 \times 3$ image information into the $1 \times 1 \times 1664$ feature map. If we use 4x scale-transfer layer, $1 \times 1 \times 1664$ feature map will be converted to $4 \times 4 \times 104$. This will increase the number of anchor boxes by 16 times, and fully explore the information contained in channels. Compared with other upscaling operation (deconvolution), there is no increase in parameters and computation. In addition, due



(a) Single feature map (b) Feature pyramid network

(c) Pyramidal feature hierarchy (d) Scale-transfer module

Figure 1. (a) Using only single scale features for prediction. (b) Integrating information from high-level and low-level feature maps. (c) Producing predictions from feature maps of different scales. (d) Our Scale-Transfer Module. It can be embedded directly into a DenseNet to obtain feature maps at different scales.

Zhou P, Ni B, Geng C, et al. Scale-Transferrable Object Detection[C]. computer vision and pattern recognition, 2018: 528-537.

Rethinking

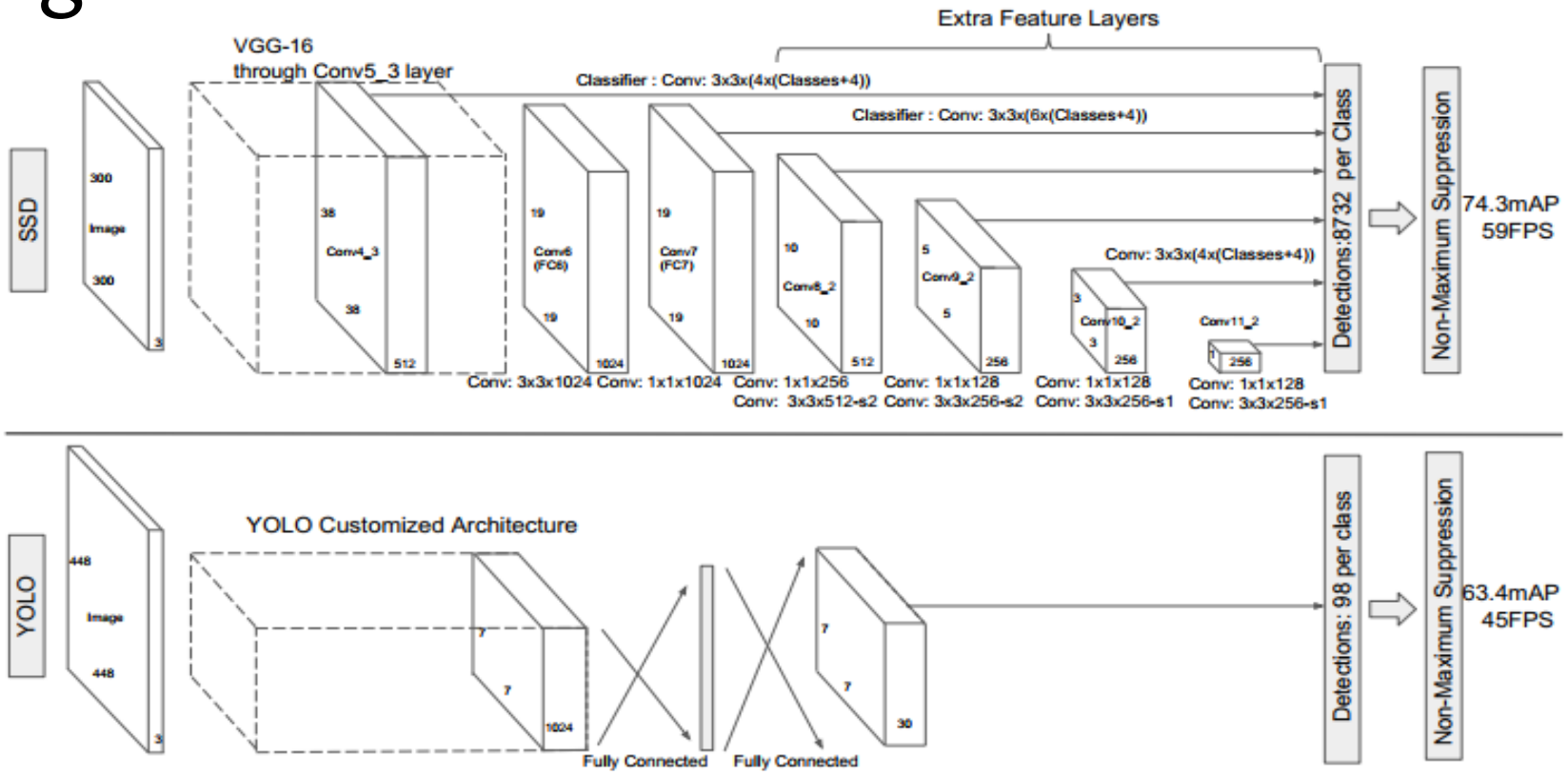


Fig. 2: A comparison between two single shot detection models: SSD and YOLO [5]. Our SSD model adds several feature layers to the end of a base network, which predict the offsets to default boxes of different scales and aspect ratios and their associated confidences. SSD with a 300×300 input size significantly outperforms its 448×448 YOLO counterpart in accuracy on VOC2007 `test` while also improving the speed.

Rethinking

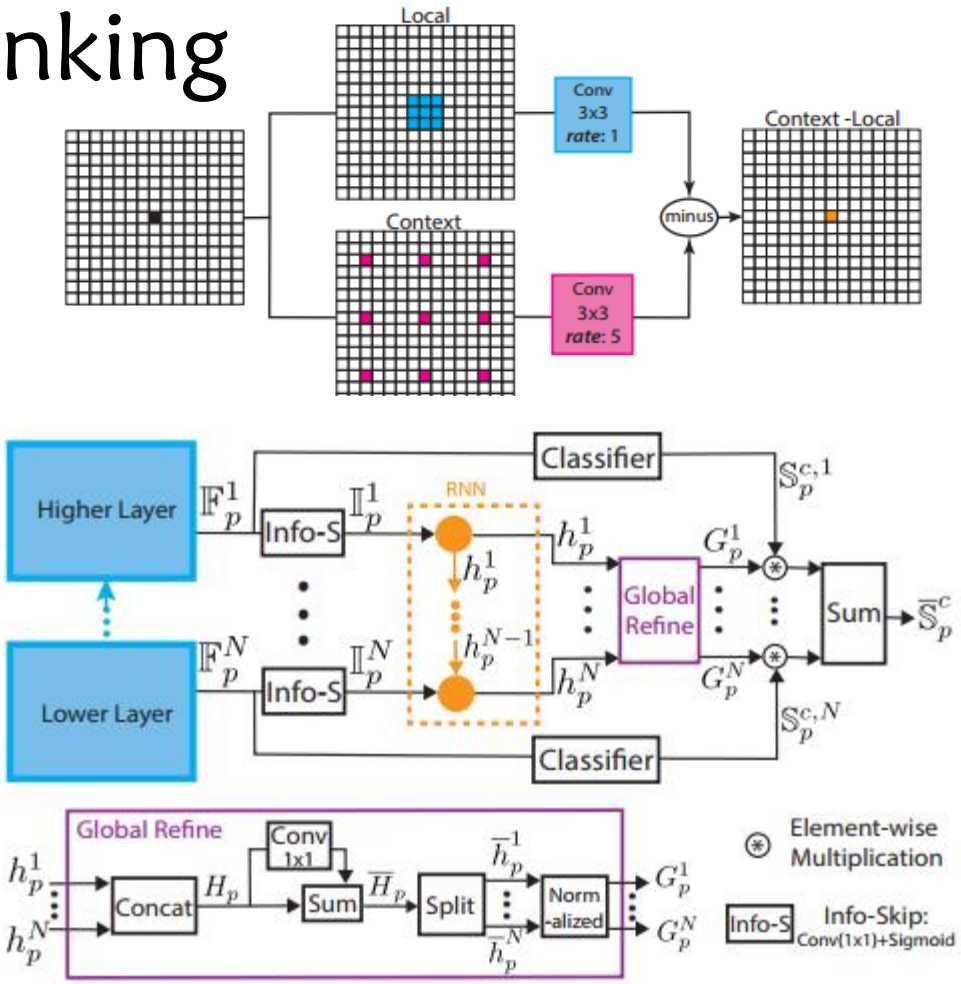


Figure 5: Gated Sum could control the information flow of skip layers via its inherent gates. The gates G_p^n could adjust its value according to the input images. All the maps in gated sum have the same spatial size of $H \times W$.

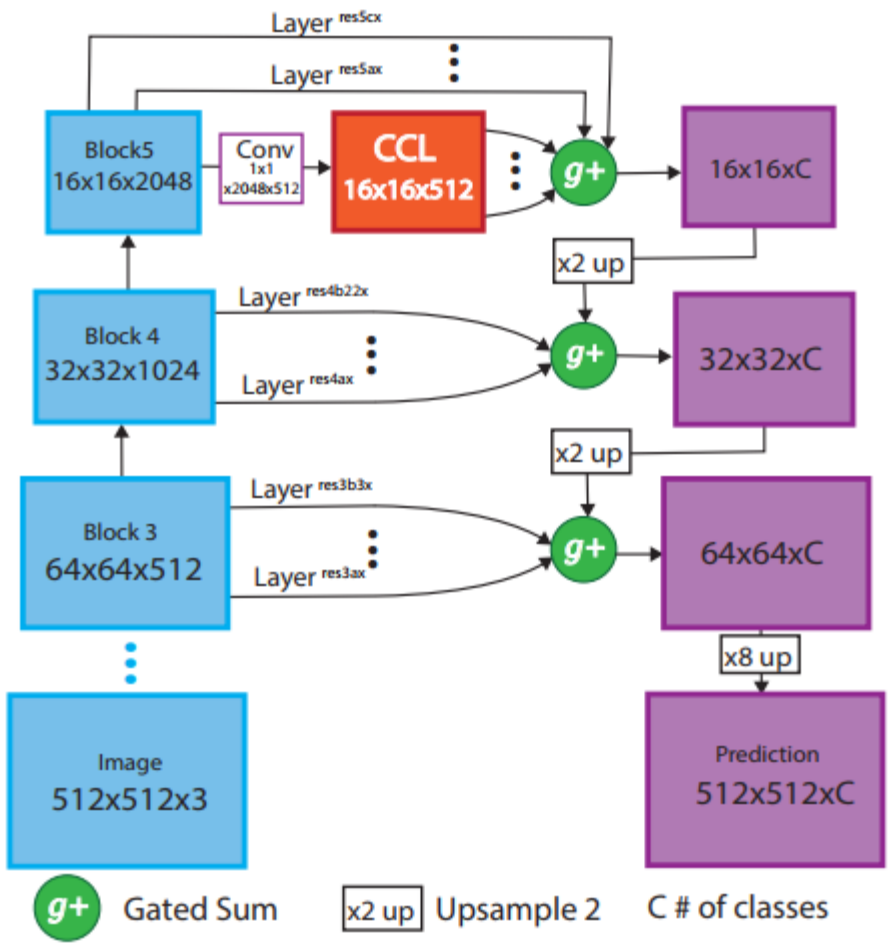


Figure 2: Overview of our network framework. The proposed context contrasted local (CCL) model generates multi-level and multi-scale context aware local features. Gated sum selectively aggregate rich scale features in DCNN and CCL.

Ding H, Jiang X, Shuai B, et al. Context Contrasted Feature and Gated Multi-Scale Aggregation for Scene Segmentation[C]. computer vision and pattern recognition, 2018: 2393-2402.